Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Гаврилюк Г.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 25.10.26

Москва, 2026

Постановка задачи

Цель работы

Приобретение практических навыков в управлении потоками в ОС и обеспечении синхронизации между потоками.

Задание

Составить программу на языке C/C++, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (POSIX Threads). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы. Необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Стандартная библиотека C++ (кроме std::vector и std::string) не используется: потоки создаются только через POSIX Threads, вывод выполняется системным вызовом write. Реализованы две версии алгоритма – последовательная и параллельная, время выполнения замеряется для обеих.

Вариант 8.

Есть K массивов одинаковой длины. Необходимо сложить эти массивы. Необходимо предусмотреть стратегию, адаптирующуюся под количество массивов и их длину (по количеству операций).

Общий метод и алгоритм решения

Использованные системные вызовы и функции POSIX:

- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg); – создаёт новый поток, который начинает выполнение с функции start_routine с аргументом arg; идентификатор потока записывается в thread.
- int pthread_join(pthread_t thread, void **retval); – блокирует вызывающий поток до завершения потока thread.
- int clock_gettime(clockid_t clk_id, struct timespec *tp); – получает текущее значение монотонных часов CLOCK_MONOTONIC, используется для замера времени.
- long sysconf(int name); – с параметром _SC_NPROCESSORS_ONLN возвращает число доступных логических процессоров (значение по умолчанию для ключа -t).
- ssize_t write(int fd, const void *buf, size_t count); – вывод результатов в стандартный поток вывода (fd = 1).
- unsigned int sleep(unsigned int seconds); – приостанавливает поток на заданное число секунд; используется в демонстрационном режиме, чтобы потоки успели отобразиться в ps.
- pid_t getpid(void); – возвращает PID процесса, чтобы найти его в выводе ps.

Метрики

Для оценки параллельной версии используются две метрики.

Ускорение показывает, во сколько раз параллельная версия быстрее последовательной: $S = T_s / T_p$, где T_s – время последовательной версии, T_p – время параллельной версии на p потоках. Идеальное значение $S = p$; $S < 1$ означает, что параллельная версия медленнее последовательной.

Эффективность показывает, какая доля идеального ускорения достигнута, то есть насколько полезно используется каждый поток: $E = S / p$, где p – число фактически запущенных рабочих потоков. Идеальное значение $E = 1$.

Верхнюю границу ускорения задаёт закон Амдала: $S \le 1 / (a + (1 - a) / p)$, где a – доля последовательной (непараллелизуемой) части работы. Помимо этого ускорение ограничено числом ядер и пропускной способностью памяти.

Алгоритм

Программа получает ключами запуска число массивов K (-k), длину массива N (-n), максимальное число потоков T (-t, по умолчанию – число логических ядер) и флаг демонстрационного режима -p. Все K массивов хранятся в одном непрерывном векторе длины $K \cdot N$: элемент j массива i лежит по индексу $i \cdot N + j$. Такое размещение даёт последовательный доступ к памяти и эффективную работу кэша. Массивы заполняются случайными числами от 0 до 99. Результат – вектор длины N, где $res[j] = a_{0,j} + a_{1,j} + \ldots + a_{K-1,j}$.

Последовательная версия – функция sum_range, которая складывает диапазон столбцов [j_begin, j_end): внешний цикл идёт по массивам, внутренний – по столбцам, поэтому данные читаются подряд. Для последовательной версии вызывается на всём диапазоне [0, N).

Параллельная версия использует модель Fork & Join. Параметры для каждого потока упаковываются в структуру ThreadArgs (указатели на данные и результат, K, N, границы куска); у каждого потока своя структура, которая живёт до завершения потока. Потоки создаются в одном цикле через pthread_create, а затем в отдельном цикле ожидаются через pthread_join – так все потоки работают одновременно. Функция потока приводит аргумент void* к ThreadArgs* и выполняет свою часть работы. Синхронизация при вычислениях не нужна: каждый поток пишет только в свою область памяти, а общие входные данные только читаются.

Работа делится между потоками **по столбцам**: поток p получает столбцы $[\lfloor p \cdot N/T \rfloor, \lfloor (p+1) \cdot N/T \rfloor)$ и пишет сразу в общий результат. Куски не пересекаются и вместе покрывают все столбцы, поэтому гонки данных нет.

Адаптивная стратегия (функция choose_threads) выбирает число потоков по количеству операций $W = K \cdot N$:

- каждому потоку должно достаться не меньше 50 000 элементов, иначе накладные расходы на создание и ожидание потока сопоставимы с полезной работой, поэтому число потоков не больше W / 50 000;
- число потоков не больше значения ключа -t (ограничение по заданию) и не больше числа столбцов N, чтобы у каждого потока был хотя бы один столбец;
- если в итоге выбран один поток (малый объём работы), потоки не создаются и сложение выполняется в главном потоке.

Замер времени. Время измеряется через clock_gettime(CLOCK_MONOTONIC) только вокруг вызова сложения; генерация данных в замер не входит. Программа выводит время последовательной и параллельной версий, ускорение и эффективность.

Демонстрация потоков (-p). Программа печатает свой PID и запускает параллельную версию, в которой каждый рабочий поток перед работой засыпает на 15 секунд. За это время число потоков процесса можно посмотреть командой ps -o nlwp= -p <PID> (выводит число потоков процесса, включая главный).

Для сборки написан Makefile (make, make run, make clean).

Код программы

main.cpp

```
#include <unistd.h>
#include <pthread.h>
#include <cstdint>
#include <cstdlib>
#include <cstring>
#include <ctime>
#include <vector>
#include <cstdio>

using std::vector;

const size_t MIN_WORK_PER_THREAD = 50000;

bool g_pause = false;



void sum_range(const vector<int64_t>& data, vector<int64_t>& res,
               size_t k, size_t n, size_t j_begin, size_t j_end) {
    for (size_t j = j_begin; j < j_end; j++) {
        res[j] = 0;
    }

    for (size_t i = 0; i < k; i++) {
        for (size_t j = j_begin; j < j_end; j++) {
            res[j] += data[i * n + j];
        }
    }
}



double elapsed_ms(timespec start, timespec end) {
    double sec  = static_cast<double>(end.tv_sec)  - static_cast<double>(start.tv_sec);
    double nsec = static_cast<double>(end.tv_nsec) - static_cast<double>(start.tv_nsec);
    return sec * 1000.0 + nsec / 1000000.0;
}



struct ThreadArgs {
    const vector<int64_t>* data;
    vector<int64_t>* res;
    size_t k;
    size_t n;
    size_t j_begin;
    size_t j_end;
};
```

```
void* worker(void* args) {
    ThreadArgs* p = static_cast<ThreadArgs*>(args);
    if (g_pause) {
        sleep(15);
    }
    sum_range(*p->data, *p->res, p->k, p->n, p->j_begin, p->j_end);
    return nullptr;
}



size_t choose_threads(size_t k, size_t n, size_t t) {
    size_t threads = k * n / MIN_WORK_PER_THREAD;
    if (threads > t) {
        threads = t;
    }
    if (threads > n) {
        threads = n;
    }
    if (threads < 1) {
        threads = 1;
    }
    return threads;
}



void sum_parallel(const vector<int64_t>& data, vector<int64_t>& res,
                  size_t k, size_t n, size_t t) {
    if (t == 1) {
        sum_range(data, res, k, n, 0, n);
        return;
    }

    vector<pthread_t> threads(t);
    vector<ThreadArgs> thread_args(t);

    for (size_t p = 0; p < t; p++) {
        thread_args[p].data = &data;
        thread_args[p].res = &res;
        thread_args[p].k = k;
        thread_args[p].n = n;
        thread_args[p].j_begin = p * n / t;
        thread_args[p].j_end = (p + 1) * n / t;

        pthread_create(&threads[p], nullptr, worker, &thread_args[p]);
    }

    for (size_t p = 0; p < t; p++) {
        pthread_join(threads[p], nullptr);
    }
}



int main(int argc, char** argv) {

    size_t k = 0;
    size_t n = 0;
    size_t t = 0;

    for (int i = 1; i < argc; i++) {
```

```
        if (strcmp(argv[i], "-k") == 0 && i + 1 < argc) {
            k = strtoull(argv[i + 1], nullptr, 10);
        }

        if (strcmp(argv[i], "-n") == 0 && i + 1 < argc) {
            n = strtoull(argv[i + 1], nullptr, 10);
        }

        if (strcmp(argv[i], "-t") == 0 && i + 1 < argc) {
            t = strtoull(argv[i + 1], nullptr, 10);
        }

        if (strcmp(argv[i], "-p") == 0) {
            g_pause = true;
        }
    }

    if (t == 0) {
        t = static_cast<size_t>(sysconf(_SC_NPROCESSORS_ONLN));
    }

    srand(time(nullptr));

    vector<int64_t> data(k * n);
    vector<int64_t> res_seq(n);
    vector<int64_t> res_par(n);

    for (size_t i = 0; i < data.size(); i++) {
        data[i] = rand() % 100;
    }

    size_t used = choose_threads(k, n, t);

    char buf[128];
    int len;

    if (g_pause) {
        len = snprintf(buf, sizeof(buf), "PID: %d, threads: %zu\n",
static_cast<int>(getpid()), used);
        write(1, buf, len);
        sum_parallel(data, res_par, k, n, used);
        return 0;
    }

    timespec start;
    timespec end;

    clock_gettime(CLOCK_MONOTONIC, &start);
    sum_range(data, res_seq, k, n, 0, n);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_seq = elapsed_ms(start, end);

    clock_gettime(CLOCK_MONOTONIC, &start);
    sum_parallel(data, res_par, k, n, used);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_par = elapsed_ms(start, end);

    double speedup = time_seq / time_par;
    double efficiency = speedup / static_cast<double>(used);
```

```
    len = snprintf(buf, sizeof(buf), "K=%zu N=%zu T(max)=%zu threads used=%zu\n", k, n, t,
used);
    write(1, buf, len);

    len = snprintf(buf, sizeof(buf), "Sequential: %.3f ms\n", time_seq);
    write(1, buf, len);

    len = snprintf(buf, sizeof(buf), "Parallel:   %.3f ms\n", time_par);
    write(1, buf, len);

    len = snprintf(buf, sizeof(buf), "Speedup:    %.3f\n", speedup);
    write(1, buf, len);

    len = snprintf(buf, sizeof(buf), "Efficiency: %.3f\n", efficiency);
    write(1, buf, len);

    return 0;
}
```

Makefile

```
CXX = g++
CXXFLAGS = -O2 -Wall -pthread

lab: main.cpp
    $(CXX) $(CXXFLAGS) main.cpp -o lab

run: lab
    ./lab -k 100 -n 1000000 -t 16

clean:
    rm -f lab
```

Протокол работы программы

Характеристики системы: Intel Core i5-1240P (4 производительных P-ядра с Hyper-Threading и 8 энергоэффективных E-ядер, всего 16 логических процессоров), кэш L3 12 МБ, WSL2 (Ubuntu), компилятор g++ с оптимизацией -O2.

Сборка:

```
$ make
g++ -O2 -Wall -pthread main.cpp -o lab
```

Пример запуска:

```
$ ./lab -k 100 -n 1000000 -t 16
K=100 N=1000000 T(max)=16 threads used=16
Sequential: 159.311 ms
Parallel:   20.936 ms
Speedup:    7.609
Efficiency: 0.476
```

Демонстрация количества потоков:

```
$ ./lab -k 100 -n 1000000 -t 16 -p
PID: <PID>, threads: 16

$ ps -o nlwp= -p <PID>
```

Команда ps -o nlwp= -p <PID> выводит число потоков (LWP) процесса. Программа запускает 16 рабочих потоков, поэтому ps показывает 17: 16 рабочих потоков плюс главный поток main, который ожидает их в pthread_join.

Исследование ускорения и эффективности

Эксперименты проводились на массиве K = 100, N = 1 000 000 (объём данных 800 МБ). Число логических ядер системы – 16. Для каждого значения ключа -t программа запускалась отдельно.

Число потоков (-t)	Запущено потоков	Время исполнения (мс)	Ускорение	Эффективность
1	1	157,980	0,991	0,991
2	2	92,665	1,707	0,854
4	4	50,549	3,129	0,782
8	8	22,676	6,907	0,863
12	12	26,585	5,980	0,498
16	16	20,936	7,609	0,476
64	64	22,100	7,271	0,114
128	128	24,454	6,497	0,051
1024	1024	142,142	1,056	0,001

Таблица 1 – зависимость времени, ускорения и эффективности от числа потоков (T_s в запусках 150–161 мс)

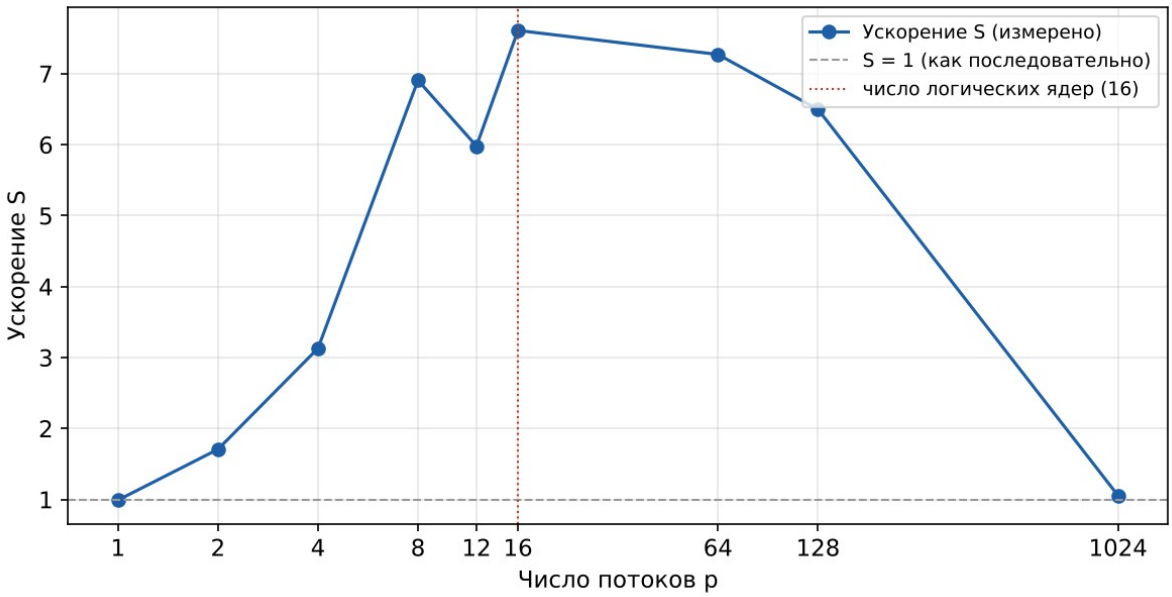


Рисунок 1 – ускорение в зависимости от числа потоков

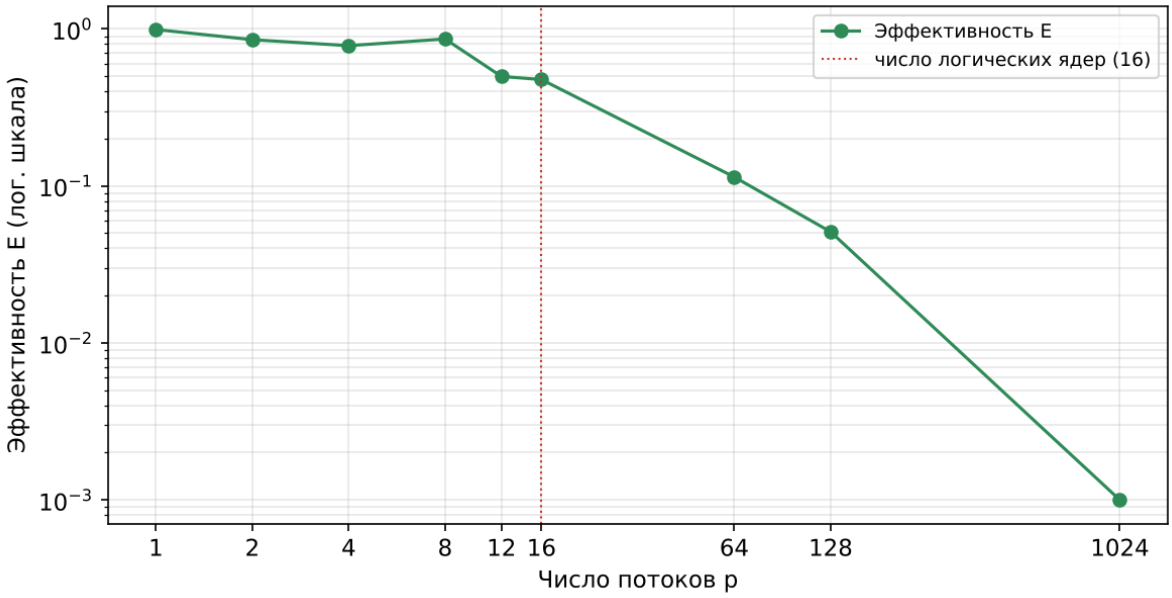


Рисунок 2 – эффективность в зависимости от числа потоков

Зависимость от входных данных (лимит потоков -t = 16):

K	N	Запущено потоков	T_s, мс	T_p, мс	S
2	1 000	1	0,002	0,002	1,071
1 000 000	8	8	10,222	17,011	0,601
4	25 000 000	16	173,252	44,543	3,890

Таблица 2 – работа адаптивной стратегии на разных входных данных

Вывод

В ходе работы я научился создавать и ожидать потоки средствами POSIX Threads (pthread_create, pthread_join), передавать потоку несколько параметров через структуру и указатель void*, а также организовывать данные так, чтобы потоки не мешали друг другу: каждый поток пишет только в свой диапазон столбцов результата, а общие входные данные только читает, поэтому синхронизация при вычислениях не нужна и гонки данных нет.

При одном потоке ускорение равно 0,99: параллельная версия выполняет тот же код в главном потоке, разница находится в пределах погрешности замера. До 8 потоков ускорение растёт почти линейно (1,71 на 2 потоках, 3,13 на 4, 6,91 на 8), а эффективность держится на уровне 0,78–0,86. Это объясняется тем, что при K = 100 и N = 1 000 000 кусок результата, который обрабатывает каждый поток, помещается в кэш ядра, и потоки в основном читают входные данные последовательно, не мешая друг другу.

На 12 потоках ускорение снижается до 5,98. Процессор i5-1240P гибридный: 4 быстрых P-ядра (с Hyper-Threading – 8 логических процессоров) и 8 более медленных E-ядер. Работа делится между потоками поровну, а pthread_join ждёт самого медленного потока, поэтому потоки, попавшие на E-ядра или делящие одно физическое P-ядро, задерживают общий результат. Максимальное ускорение 7,61 получено на 16 потоках, когда заняты все логические процессоры, но эффективность при этом лишь 0,48: физических ядер всего 12, два логических процессора одного P-ядра делят его вычислительные блоки, а все ядра делят общую шину памяти, пропускная способность которой ограничивает задачу сложения массивов.

При числе потоков больше числа логических ядер наблюдается псевдопараллелизм. На 64 и 128 потоках ускорение уже не растёт и снижается (7,27 и 6,50): потоки ждут очереди на ядра, ОС тратит время на переключение контекста, а создание и ожидание каждого потока добавляет накладные расходы. Эффективность падает до 0,11 и 0,05, так как число потоков p растёт, а ускорение – нет. На 1024 потоках ускорение почти пропадает (1,06): каждому потоку достаётся около 100 000 элементов, и время на создание и ожидание тысячи потоков сравнимо со временем всей полезной работы.

Исследование на разных входных данных показывает работу адаптивной стратегии и её ограничения. При K = 2, N = 1000 работы мало (2000 элементов), поэтому стратегия выбирает один поток и не тратит время на создание потоков – ускорение около 1. При K = 4, N = 25 000 000 ускорение 3,89, заметно меньше, чем при K = 100: массивов мало, поэтому основную долю обращений к памяти составляет чтение и запись самого результата, а кусок результата каждого потока (около 12 МБ) уже не помещается в кэш. При K = 1 000 000, N = 8 столбцов всего 8, поэтому стратегия ограничивает число потоков восемью, но весь результат занимает 64 байта – одну кэш-линию. Потоки пишут в соседние элементы одной кэш-линии, ядра постоянно передают её друг другу (false sharing), и параллельная версия оказывается медленнее последовательной (S = 0,60). Для такой формы данных было бы выгоднее делить работу не по столбцам, а по массивам, с отдельными частичными суммами у каждого потока.

Таким образом, ускорение ограничено числом и типом ядер, пропускной способностью памяти и накладными расходами на потоки; наилучший результат достигается при числе потоков, близком к числу логических ядер, и достаточно большом объёме работы на каждый поток, а чрезмерное число потоков и неудачное разбиение данных снижают ускорение и эффективность.